

Guía de Transición Rápida: Java a Kotlin

Módulo: Programación Multimedia y Dispositivos Móviles (PMDM)

Orientado a: Alumnado con base en Java

Enfoque: Referencia Rápida e Idiomática

Nota del profesorado: Como ya conocéis la lógica de programación orientada a objetos de Java, no necesitamos aprender a programar de nuevo. Esta guía os servirá para traducir de forma rápida conceptos y patrones de Java a la sintaxis limpia y moderna de Kotlin.

01 Sintaxis Básica y Variables

En Kotlin **no se usan punto y coma (;)**. La declaración de tipos se realiza a la derecha y el compilador infiere tipos automáticamente siempre que exista asignación inicial.

- `val`: Variable inmutable (equivalente a `final` en Java). **Opción por defecto.**
- `var`: Variable mutable (se puede reasignar). Usar solo cuando sea estrictamente necesario.

```
// Java: final String nombre = "Juan";  
val nombre = "Juan"  
  
// Java: int edad = 20; edad = 21;  
var edad = 20  
edad = 21
```

02 Seguridad frente a Nulos (Null Safety)

Kotlin elimina los errores `NullPointerException` (NPE) en tiempo de compilación. Por defecto, ninguna variable puede almacenar un valor nulo a menos que se declare explícitamente como nula añadiendo `?`.

```
var texto: String = "Hola"  
// texto = null --> ERROR de compilación  
  
var textoNulo: String? = "Hola"  
textoNulo = null // Permitido  
  
// Safe call (?.): Si es null, la expresión devuelve null sin lanzar excepción  
val longitud: Int? = textoNulo?.length  
  
// Operador Elvis (?:): Proporciona un valor por defecto si la parte izquierda es null  
val longitudSegura: Int = textoNulo?.length ?: 0
```

03 Estructuras de Control e Interpolación

Olvidate de concatenar cadenas con `+` o usar `String.format()`. Kotlin soporta plantillas de texto nativas. Además, estructuras como `when` reemplazan con mayor potencia a `switch`.

```
// Interpolación de cadenas (String Templates)
val usuario = "Ana"
println("Bienvenida, $usuario. Tu nombre tiene ${usuario.length} letras.")

// Expresión 'when' (evalúa rangos, condiciones o valores)
val nota = 8
val resultado = when (nota) {
    in 0..4 -> "Suspenso"
    5, 6 -> "Aprobado"
    7, 8 -> "Notable"
    9, 10 -> "Sobresaliente"
    else -> "Nota no válida"
}
```

04 Funciones y Parámetros

Las funciones se declaran con `fun`. Admiten valores por defecto para evitar la sobrecarga masiva de métodos típica de Java.

```
// Función tradicional
fun sumar(a: Int, b: Int): Int {
    return a + b
}

// Función de expresión única (retorno implícito)
fun multiplicar(a: Int, b: Int) = a * b

// Valores por defecto (evita crear múltiples sobrecargas en Java)
fun saludar(nombre: String, mensaje: String = "Bienvenido") {
    println("$mensaje, $nombre")
}
```

05 Clases y Data Classes

El constructor primario se define en la propia cabecera de la clase. Las `data class` generan automáticamente `equals()`, `hashCode()`, `toString()` y `copy()`, reemplazando los verbosos POJOs de Java.

```
// Clase estándar con constructor primario
class Persona(val nombre: String, var edad: Int)

// Equivale a un JavaBean/POJO de 50 líneas en Java
data class Usuario(val id: Long, val email: String)

// Ejemplo de uso:
val user = Usuario(1, "ana@email.com")
println(user) // Imprime: Usuario(id=1, email=ana@email.com)
```

06 Colecciones y Programación Funcional

En Kotlin las colecciones son ****inmutables por defecto**** (`listOf`, `mapOf`). Para modificarlas se usa `mutableListOf`.

```
val frutas = listOf("Manzana", "Plátano", "Naranja")

// Programación funcional con lambdas ('it' representa el elemento actual)
frutas.filter { it.startsWith("M") }
    .map { it.uppercase() }
    .forEach { println(it) }
```

07 Tabla Equivalencias Rápidas Java vs Kotlin

CONCEPTO / OPERACIÓN	EN JAVA	EN KOTLIN
Variable Inmutable	<code>final String s = "Hola";</code>	<code>val s = "Hola"</code>
Variable Mutable	<code>int x = 10;</code>	<code>var x = 10</code>
Impresión en Consola	<code>System.out.println(x);</code>	<code>println(x)</code>
Control Multicondición	<code>switch (x) { ... }</code>	<code>when (x) { ... }</code>
Declaración de Método/ Función	<code>void miMetodo() { ... }</code>	<code>fun miMetodo() { ... }</code>
Clase de Datos (POJO)	Clase con getters, setters, toString...	<code>data class Usuario(val id: Int, val nom: String)</code>
Comprobación de Tipo (Casting)	<code>if (obj instanceof String) (String)obj</code>	<code>if (obj is String) obj.length (Smart Cast)</code>
Valor Nulo Seguro	<code>if (x != null) x.getFoo()</code>	<code>x?.getFoo()</code>